Merrill & Sabharwal on depth: what constant depth cannot do, what log depth can

Two papers, one argument

(1) **The Parallelism Tradeoff: Limitations of Log-Precision Transformers** — TACL 2023, arXiv 2207.00729
(2) **A Little Depth Goes a Long Way: The Expressive Power of Log-Depth Transformers** — NeurIPS 2025, arXiv 2503.03961

William Merrill (NYU / Ai2) · Ashish Sabharwal (Ai2)

Reading companion · cited in thesis ch. 1.4 as merrill2023, merrill2025 · checked against the thesis 2026-10-22

★ TL;DR

2023: a transformer of **fixed depth** whose numbers have $O(\log n)$ bits can be simulated by a constant-depth, polynomial-size **threshold circuit** that one simple program can build for every n — the class **uniform $\mathbf{TC^0}$**. So such a transformer can solve nothing harder than TC^0. Graph connectivity is believed to lie outside TC^0 (it is complete for log-space, L), so fixed-depth transformers should fail on it for large enough n. **2025:** let the depth grow just a little, $\Theta(\log n)$, by looping a block of layers, and a **fixed-width** transformer can decide connectivity (Theorem 2) and recognise regular languages (Theorem 1). Growing **width** instead does not help unless it grows faster than any polynomial (Theorem 3), and $O(\log n)$ chain-of-thought steps do not help either (Theorem 4). Experiments on a hard regular language confirm the shape: needed depth grows like $\log n$, needed width like $2^{\Theta(n)}$.

✦ The big question

How much does **depth** buy a transformer? Which problems are out of reach at fixed depth, and how fast must depth (or width, or CoT) grow to reach them?

✦ The one-line answer

Fixed depth = TC^0 (very parallel, shallow). A logarithmic amount of depth escapes it; width alone essentially cannot.

0 · Background you need

▸ Definition — Boolean circuit, depth, size

A directed acyclic graph of gates computing a function of n input bits. **Size** = number of gates; **depth** = longest input-to-output path. A **circuit family** $C_1, C_2, \ldots$ has one circuit per input length.

▸ Definition — TC^d

Problems solved by circuit families of **constant depth** and **polynomial size** built from AND, OR, NOT and **threshold** gates ("are at least k of my inputs 1?"), each with any number of inputs. Threshold gates make **counting and adding** cheap: adding n numbers, multiplication, majority and sorting are all in TC^0.

▸ Definition — Uniformity

In a **non-uniform** family each C_n can be arbitrary — it may even hard-code answers to undecidable questions, one per n. A **uniform** family must be produced by one simple program (here: a Turing machine using $O(\log n)$ memory) on input n. Uniform classes are the realistic ones, comparable with P, NP, L.

▸ Definition — The classes around it

$\mathrm{AC}^0 \subsetneq \mathrm{TC0} \subseteq \mathrm{NC}^1 \subseteq \mathrm{L} \subseteq \mathrm{NL} \subseteq \mathrm{P}$. Undirected connectivity is L-complete (Reingold 2008); directed reachability is NL-complete. Most researchers believe $\mathrm{TC}^0 \neq \mathrm{NC}^1$ (hence $\mathrm{TC}^0 \neq \mathrm{L}$), but none of the strict separations after AC^0 is proved.

My note

The chain of reasoning in both papers is: **transformers of fixed depth ⊆ TC^0**; **connectivity ∉ TC^0 (conjectured)**; therefore fixed-depth transformers cannot decide connectivity for **all** n. Every conclusion inherits the "conjectured" — and the "for all n".

Part I · The Parallelism Tradeoff (TACL 2023)

1 · The formal model (§4)

▸ **Definition — p-precision transformer**

Every value — inputs, hidden states, attention scores, outputs of every operation — is a string of p bits, and every function inside a layer (similarity s, the combining function f that stands for residual + layer-norm + feedforward) must be computable by a Turing machine using p bits of space. Summing n numbers is done approximately, like real floating point (App. A). Depth d is **fixed** as n grows. **Log-precision**: $p = c \log n$.

Why $\log n$? It is the least precision that can represent positions $1...n$ (so a token can "point" at another), and it rules out the unrealistic trick of packing the whole input into one number. The space condition on f stops "hiding" extra precision inside the feedforward net.

My note

In this 2023 model a token's **whole state** is p bits — a constant number of $O(\log n)$-bit values. The extension to **polynomial width** (and to padding tokens) is stated as Theorem 3 of the 2025 paper, "which extends Merrill and Sabharwal (2023a)".

2 · The main results (§5–6)

◆ **Theorem 1 / Corollary 1.1 (non-uniform)**

Any $c \log n$-precision transformer of depth d can be simulated by a threshold circuit family of depth $3 + (9 + 2d_{\oplus})d$ — constant in n — and polynomial size.

◆ **Theorem 2 / Corollary 2.1 (uniform, main result)**

The same holds with a **logspace-uniform** threshold circuit family: log-precision transformers are in **uniform TC^0**.

Proof idea, in three facts.

1. Any function of $c \log n$ bits has only $2^{c \log n} = n^c$ possible inputs, so it is a lookup table: a polynomial-size, depth-3 AND/OR circuit (Lemma 1). This covers the similarity score, the feedforward net, layer-norm — everything applied to one or two tokens.
2. Attention needs the sum of n numbers of $c \log n$ bits (for the normaliser Z_i and the weighted value sum). Iterated addition is in TC^0 (Lemma 5, App. A).
3. Each layer is therefore a constant-depth circuit; a fixed number of layers stacks to a constant-depth circuit. Uniformity (Lemma 2, Theorem 2) shows a log-space program can write the circuit down, by looping over the n^c table entries with an $O(\log n)$-bit counter.

3 · What follows (§2)

If a problem is complete for a class believed larger than TC^0, log-precision transformers cannot solve it exactly for all n. Under $TC^0 \neq P$: linear equalities, universal context-free recognition, Horn-SAT; under $TC^0 \neq NP$: SAT. And in the other direction: "the computation performed by such transformers is, for instance, **no harder than solving basic L-complete problems like graph connectivity**" — $TC^0 \subseteq L$.

The authors call this the **parallelism tradeoff**: transformers are in TC^0 **because** they are built to be massively parallel, and the same parallelism that makes them trainable at scale limits what they can compute.

> **My note**
>
> Two caveats the authors state themselves (§2): the results are **asymptotic** — "it's possible for log-precision transformers to solve such problems easily when n is small" — and they concern **exact** solutions. Our graphs have $n \leq 56$ (72 with k-hop encodings): far from the asymptotic regime.

4 · A lower bound: following circuit "instructions" (§7)

> **◆ Theorem 3**
>
> Depth-$2d$ transformers can solve the circuit value problem for depth-d TC^0 circuits: given a serialised threshold circuit C and an input x, output $C(x)$.

So TC^0 is roughly **tight**: a transformer can run any TC^0 computation if it is handed the circuit as part of its input ("instruction following"); with polynomial-length **advice** strings it covers non-uniform TC^0 (Corollary 3.2). The construction uses saturated (averaging-hard) attention and fractional positions i/n.

Part II · A Little Depth Goes a Long Way (NeurIPS 2025)

5 · The model: looped ("universal") transformers (§2)

> **▸ Definition — (s, r, t)-universal transformer**
>
> s fixed initial layers, a block of r layers **repeated** $d(n)$ times, and t final layers: $s + rd(n) + t$ layers in total, with $d(n) = \lceil \log_2 n \rceil$. Weights are fixed — they do not depend on n — and shared across repetitions. A standard depth-d transformer is the case $(d, 0, 0)$.

Modelling choices: **averaging-hard attention** (all weight split evenly over the maximal scores — a large-temperature idealisation), **masked pre-norm** (layer-norm reads a selected subset of the residual stream), residual connections, and arithmetic precise to $p = c \log n$ bits. They note that **finite-precision** transformers (only $O(1)$ correct bits) do not satisfy this.

> **My note**
>
> A looped transformer unrolled to depth d is just an ordinary depth-d transformer with repeated weights. So every result about log depth also says: a **fixed** depth d suffices for all inputs up to size $2^{O(d)}$ (Corollaries 1.2, 2.1).

6 · The tool: dividing small integers (§3, Lemma 1)

A 7-layer block computes $\lfloor a_i/m \rfloor$ and $a_i \bmod m$ at every position, with fixed weights. This is what lets a single set of weights **route information** by position arithmetic (e.g. "token p stands for the pair (i, j) with $p = in + j$") for every n — earlier log-depth constructions (Liu et al. 2023) needed weights that changed with n.

7 · Theorem 1 — regular languages at log depth (§4)

> **◆ Theorem 1**
>
> For a regular language with automaton states Q there is a $(0, 8, 9)$-universal transformer of width $O(|Q|^2)$ (or $O(|Q| \log |Q|)$ for a DFA) that recognises it when unrolled $\lceil \log_2 |w| \rceil$ times.

Mechanism: an associative prefix-"sum" of state-transition maps, combined pairwise in a binary tree — $\log n$ levels. Non-uniformly, $4\lceil \log_2 n \rceil + 5$ layers suffice (Corollary 1.1).

8 · Theorem 2 — graph connectivity at log depth (§5, App. D)

◆ Theorem 2

There is a $(17, 2, 1)$-universal transformer with **fixed** width m and fixed feedforward width w that, unrolled $\lceil \log_2 n \rceil$ times, decides connectivity on directed or undirected n-vertex graphs. Input: the $n \times n$ adjacency matrix **as n^2 one-bit tokens**, then **n^3 padding tokens**, then s and t in unary.

The algorithm is exactly **repeated squaring**, written as two predicates:

$$B_0(i,j) \Leftrightarrow A(i,j) \vee i = j, \quad C_{\ell+1}(i,k,j) \Leftrightarrow B_{\ell(i,k)} \wedge B_\ell(k,j), \quad B_{\ell+1}(i,j) \Leftrightarrow \exists k : C_{\ell+1}(i,k,j).$$

$B_\ell(i,j)$ = "a path of length $\leq 2^\ell$". One token per pair (i,j) holds B_ℓ; one padding token per **triple** (i,k,j) holds C_ℓ. Each loop: triples attend to their two pairs and AND them; pairs attend uniformly over their n triples and OR them. After $\lceil \log_2 n \rceil$ loops the final token reads $B(s,t)$.

✎ My note

Where the width went. The width is fixed, but the sequence has $N \approx n^3$ tokens — one per triple. The "$\exists k$" that a narrow model cannot do inside one token is spread over n extra tokens per pair. Theorem 3 below counts padding tokens as width, so this is not a loophole: it is **depth** that takes the problem out of TC^0.

✎ My note

Depth needed, concretely. Total depth $2\lceil \log_2 n \rceil + 18$. Corollary 2.1 states $n \leq 2^{(d-20)/2}$; the paper's prose examples ("depth 32 … graphs with up to 128 vertices", "depth 80 … 2.1B vertices") match the slightly tighter $n \leq 2^{(d-18)/2}$. Either way the construction needs **about 20 layers before it covers any graph at all**. Our 6-layer models solve $n = 56$ — with n tokens, but width growing with n.

9 · Theorems 3 and 4 — width and CoT do not substitute (§6)

◆ Theorem 3 (width scaling)

A fixed-depth transformer whose width — model dimension **or** number of padding tokens — grows at most polynomially in n, with weights computable in L, can be simulated in L-uniform TC^0.

◆ Theorem 4 (CoT scaling)

Transformers with $O(\log n)$ chain-of-thought steps recognise only languages in L-uniform TC^0.

So, for problems outside TC^0: depth $\Theta(\log n)$ suffices; width must grow **super-polynomially**; CoT must use **more than logarithmically** many steps. "Scaling depth is more efficient than scaling width or chain-of-thought steps."

10 · Experiments (§7, App. F)

- Task: **A_5 state tracking** — the running product of a sequence of even permutations of 5 elements; a canonical hard regular language (NC^1-complete).
- 100 million training sequences, lengths up to 1024. Two series: width 512 with depth 6–24; depth 6 with width 128–1024. Several seeds.
- Measured: n^* = the largest position where validation accuracy is at least 95%; the **best** seed per configuration.
- Fits: depth $= 4.8 \log_2 n - 15.8$ ($r^2 = 0.93$); context $n = 7.2 \log_2 w - 41.7$ ($r^2 = 0.98$). Depth grows logarithmically with n; width exponentially.
- 4.8 layers per doubling sits between the uniform construction (8) and the non-uniform one (4).

✎ My note

Their n^* is the mirror image of our critical width: fix the model, find the largest input it handles at 95%. Two protocol differences: they take the **best** seed (we average per-seed crossings), and their task is a regular language over sequences, not a graph. Connectivity is **not** tested experimentally.

11 · Checking our thesis against the papers

Where	What we wrote	Verdict
ch. 1.4, l. 232–234	a fixed-width transformer run $\lceil \log_2 n \rceil$ times can decide connectivity [merrill2025]	**correct but incomplete**: the construction also needs n^3 padding tokens (the adjacency matrix as n^2 one-bit tokens) and about 18 extra fixed layers
ch. 1.4, l. 234–235	constant-depth, polynomial-width transformers lie in TC^0 [merrill2023]	**cite both**: 2023 proves it for log precision with $O(\log n)$-bit token states; the polynomial-width (and padding) statement is 2025 Theorem 3, which extends 2023
ch. 1.4, l. 235–237	under accepted conjectures, constant depth needs super-polynomial width	correct ($TC^0 \neq$ L, resp. NL for directed graphs)
ch. 1.4, l. 237–241	the two results differ in assumptions: Merrill–Sabharwal count the whole transformer, MLPs included, as a polynomial-size circuit built by one rule for all n	correct (2023: f computable in $O(\log n)$ space; 2025 Thm 3: weights computable in L)
ch. 1.1, eq. (squaring)	repeated squaring is the core of the log-depth results [ref to 1.4]	correct, and 2025 Theorem 2 **is** repeated squaring — worth citing merrill2025 there

12 · Hooks into our work

💡 **Thesis hook**

- **Our regime is the escape hatch they name.** Both papers are asymptotic and about exact solutions; the 2023 paper says small n may be easy. We train at $n \leq 56$ with 32-bit floats, so the TC^0 bound does not forbid what we see; it predicts only that at **fixed** depth 6 the width needed must eventually grow faster than any polynomial. Our $n^{1.74}$ over $n = 32$–56 cannot test "eventually".
- **Depth-6 models beat the constant.** Their uniform construction needs $\approx 2\log_2 n + 18$ layers; our trained models need 6 at $n = 56$. They pay with width ($m^* \approx n$) where the construction pays with n^3 tokens. Two currencies for the same "$\exists k$".
- **A width-n picture of doubling.** With only n tokens, one natural way to square is for token i to hold its whole row $B_\ell(i, \cdot)$ — n bits — and OR the rows of the nodes it reaches. That needs width of order n at depth $\log n$, close to our $m^* \approx 0.7n$–$1.0n$. Our set probe argues against **linearly readable** rows (AUC ≤ 0.89), so if the model squares, it stores rows in a non-linear code. Worth a sentence in ch. 4 or 5.
- **Their experiment is the depth-side twin of ours.** They show the depth needed grows like $\log n$ for a regular language; our Q7a/Q7c show that, at $n = 40$, beyond 4–6 layers depth buys nothing, and at depth 2 width explodes ($\approx n^3$).

? Open question

Would a **looped** model (weights shared across layers, as in their Theorem 2) learn connectivity at a smaller width than our 6 untied layers? They list this as future work for their own experiments ("Limitations").

13 · One-paragraph summary

Merrill and Sabharwal place transformers inside circuit complexity. In 2023 they prove that a fixed-depth transformer computing with $O(\log n)$-bit numbers is a uniform TC^0 circuit — highly parallel and shallow — so, if $TC^0 \neq L$, it cannot decide graph connectivity for all graph sizes. In 2025 they show the limit is about **depth**: looping a block of a fixed-width transformer $\lceil \log_2 n \rceil$ times decides connectivity by repeated squaring (with n^3 padding tokens to hold the intermediate "path through k" facts), whereas polynomial width or $O(\log n)$ chain-of-thought steps stay in TC^0. Their experiments on A_5 state tracking match the theory: depth grows logarithmically with the input length, width exponentially. For our thesis they supply the depth side of the depth–width picture, the reason depth $\approx \log n$ is the natural choice, and a precise statement of why constant-depth claims (Yehudai et al.) need powerful MLPs to hold.